Programação 2

Aula 4 - Versionamento de Código: ferramentas e processos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Tipos e estruturas elementares de dados

Linguagens de programação definem **tipos de dados primitivos/predefinidos**

C

```c
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
```

Python

```python
caractere = 'a'
numero_inteiro = -100
numero_continuo = 13.6
valor_verdade = True
sequencia_de_chars = "teste"
```

Java

```java
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
boolean valor_verdade = true;
```

C#

```c#
char caractere = 'a';
int numero_inteiro = -100;
float numero_continuo = 13.6f;
double numero_continuo_2 = 13.6;
bool valor_verdade = true;
string sequencia_de_chars = "teste";
```

Tamanhos dos dados

- chars [8 bits ou 1 byte] = 256 possíveis valores, de 00000000 até 11111111
- bool [8 bits ou 1 byte] = 256 possíveis valores, apenas 2 explorados 00000000 ou 11111111
- int [32 bits ou 4 bytes] = de -2,147,483,648 até 2,147,483,647
- float [32 bits ou 4 bytes] = de 1.175494351 E-38 até 3.402823466 E+38
- double [64 bits ou 8 bytes] = de 2.225073858507201 4 E-308 até 1.797693134862315 8 E+308

Operadores

Linguagens de programação definem **operadores** para manipularem os dados

Aritméticos:	+	-	*	/	%	++	--
Comparação:	==	!=	>	<	>=	<=	
Lógicos:	&&	||	!				
Lógicos [Python]:	and	or	not				
Atribuição:	=	+=	-=	*=	/=	%=	
Bitwise:	&	|	^	~	<<	>>	

```
byte variavelA = 00001111; // o valor da variavelA no sistema decimal é 15
byte variavelB = 01111110; // o valor da variavelB no sistema decimal é 126
byte variavelC = variavelA & variavelB; // em bits, variavelC após essa linha é 00001110, 14
byte variavelD = variavelA ^ variavelB; // o valor de variavelD após essa linha é 01110001, 113
byte variavelE = variavelD << 1; // o valor de variavelE após essa linha é 11100010, 226
```

Instruções condicionais

Programadores podem controlar o **fluxo de execução** a partir de **condições**

C, Java, C#

```
int pontos_de_vida_iniciais = 100;
int dano_do_ataque = 17;
int pontos_de_vida_atuais = pontos_de_vida_iniciais - dano_do_ataque;

if(pontos_de_vida_atuais <= 0)
{
    // o personagem não tem mais pontos de vida
    // notificar mensagem de fim do jogo
}
else if (pontos_de_vida_atuais < 25)
{
    // o personagem tem menos que 25 pontos de vida
    // notificar o jogador para ter cuidado
}
else
{
    // notificar o jogador a quantidade de pontos perdidos
}
```

Python

```
pontos_de_vida_iniciais = 100
dano_do_ataque = 17
pontos_de_vida_atuais = pontos_de_vida_iniciais - dano_do_ataque

if pontos_de_vida_atuais <= 0:
    # o personagem não tem mais pontos de vida
    # notificar mensagem de fim do jogo

elif pontos_de_vida_atuais < 25:
    # o personagem tem menos que 25 pontos de vida
    # notificar o jogador para ter cuidado

else
    # notificar o jogador a quantidade de pontos perdidos
```

Instruções de repetição

Programadores podem controlar o **fluxo de execução** a partir de **repetições**

C, Java, C#

```
// executando a potencia com a repetição usando while
int i = 1;
int numeroA = 2;
int potencia = 5;

while (i < potencia)
{
  numeroA = numeroA * numeroA;
  i++;
}
```

```
int numeroA = 2;
int potencia = 5;
// executando a potencia com a repetição usando for
for(int i = 1; i < potencia; i++)
{
    numeroA = numeroA * numeroA;
}
```

Python

```
i = 1
numeroA = 2
potencia = 5

while i < potencia:
  numeroA = numeroA * numeroA;
  i += 1
```

Funções e Métodos

Declaração e Implementação

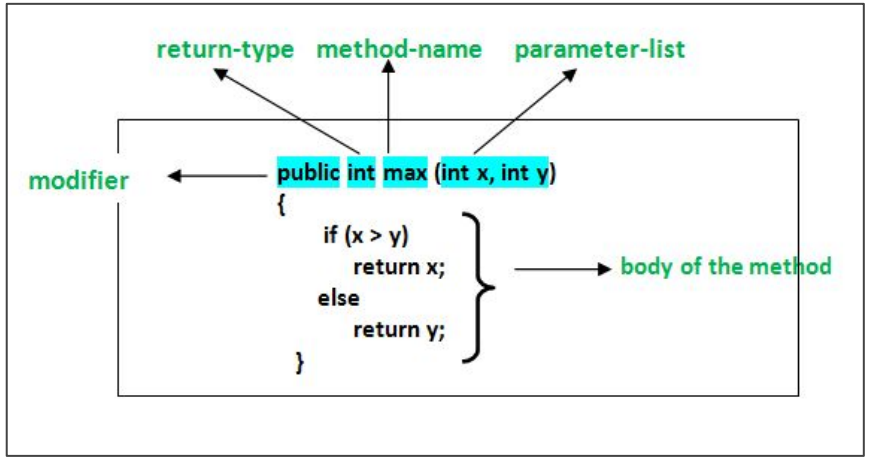
C

```c
char abc(char ch1, char ch2)
{
    char ch3;
    …
    …
    return ch3;
}
```

Python

```python
def sum(a, b):
    return a + b
```

Java, C#



Chamadas

C

```c
char resultado = abc('x', 'y');
```

Python

```python
soma = sum(129, 13)
```

Java, C#

```java
int maximo = max(7, 15);
```

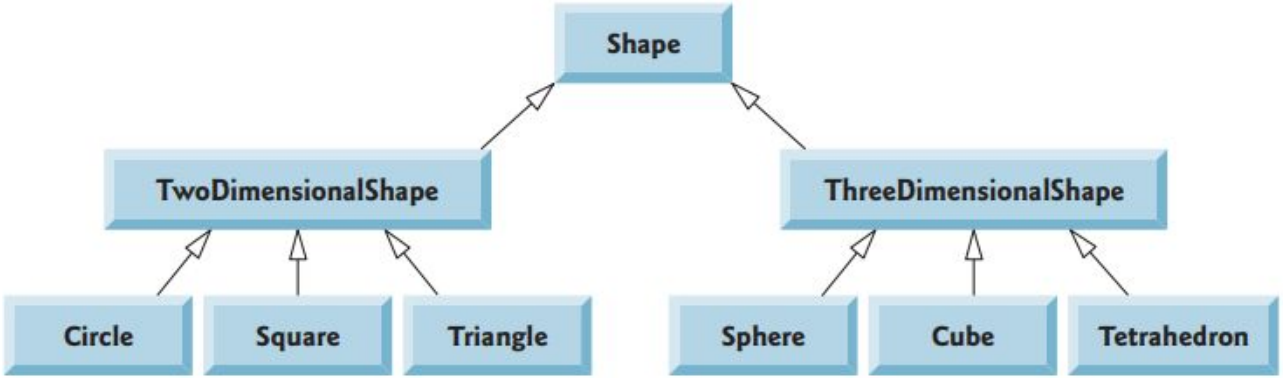
Método *main*

Permite que o programa seja executado sem a criação de nenhum objeto

A máquina virtual responsável por executar o programa Java que se chama Java Virtual Machine (**JVM**) **inicia a execução** pelo método *main*

Herança

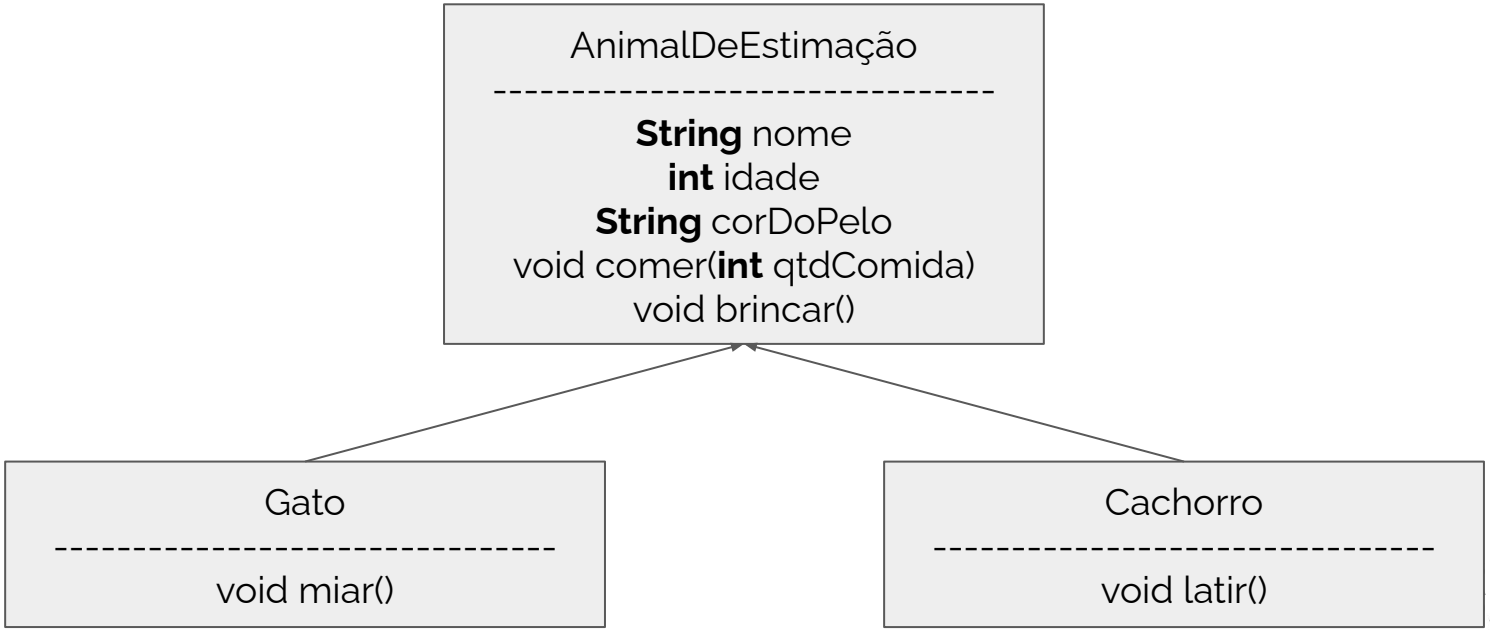
Herança é uma forma de explicitar que uma determinada classe **tem** e **faz** tudo que outra classe faz, e mais um pouco.



Atenção! *Cuidado para não confundir classes com objetos. No exemplo acima temos classes que herdam de outras classes. Para cada uma dessas classes podem existir vários objetos*

Herança

Herança é uma forma de explicitar que uma determinada classe **tem** e **faz** tudo que outra classe faz, e mais um pouco.

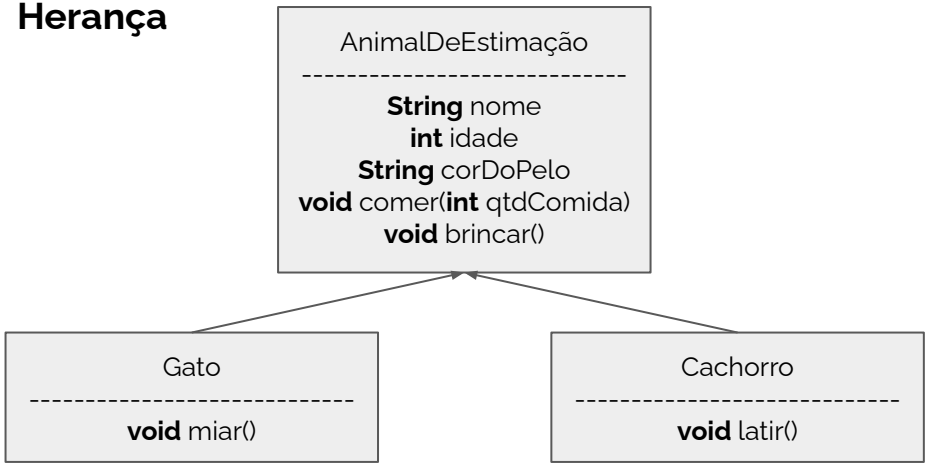


UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

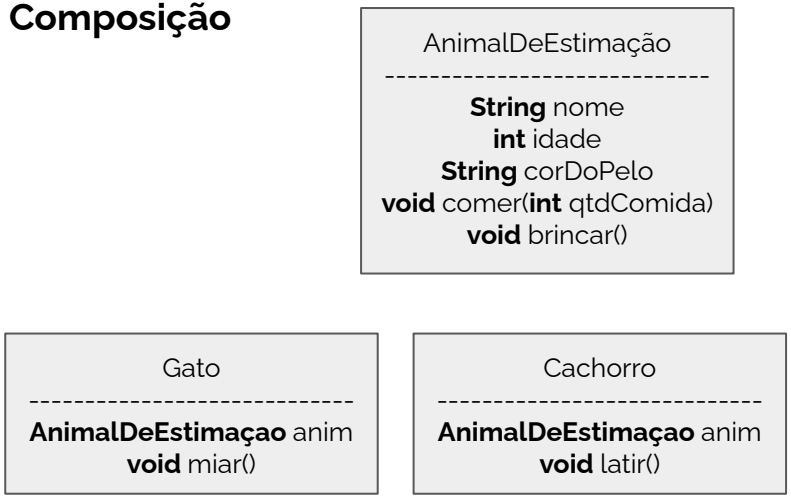
Herança x Composição

Uma opção para resolver o mesmo tipo de problema é usar a composição

Herança



Composição



Herança: SubClasse e SuperClasse

Ao criar uma classe, em vez de declarar membros completamente novos, você pode designar que a nova classe, chamada de **subclasse**,
deva herdar membros de uma classe existente, que é chamada de **superclasse**.

(A linguagem de programação C++ refere-se à superclasse como a classe básica e a subclasse como a classe derivada.) Uma subclasse pode tornar-se uma superclasse para futuras subclasses

Herança: SubClasse

Uma subclasse

- pode adicionar seus próprios campos (**atributos**) e **métodos**, portanto, ela é mais específica que sua superclasse
- exibe os comportamentos da superclasse e **pode modificá-los** de modo que eles operem adequadamente para a subclasse
- é por isso que a herança é às vezes chamada **especialização**.

Herança: Hierarquia

A **superclasse direta** é aquela a partir da qual a subclasse herda explicitamente.

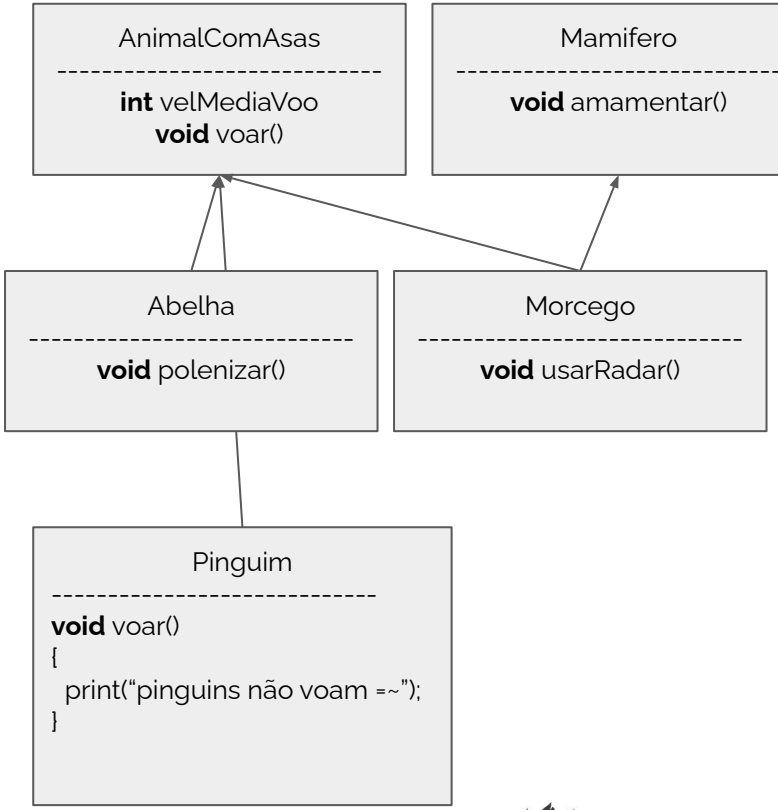
Uma **superclasse indireta** é qualquer classe acima da superclasse direta na **hierarquia de classes**, que define os relacionamentos de herança entre classes

No Java, a hierarquia de classes inicia com a classe **Object**, da qual toda classe em Java direta ou indiretamente **estende** (ou “herda de”)

Herança: Única x Múltipla

Herança única é aquela em que cada classe é derivada **exatamente de uma** superclasse direta

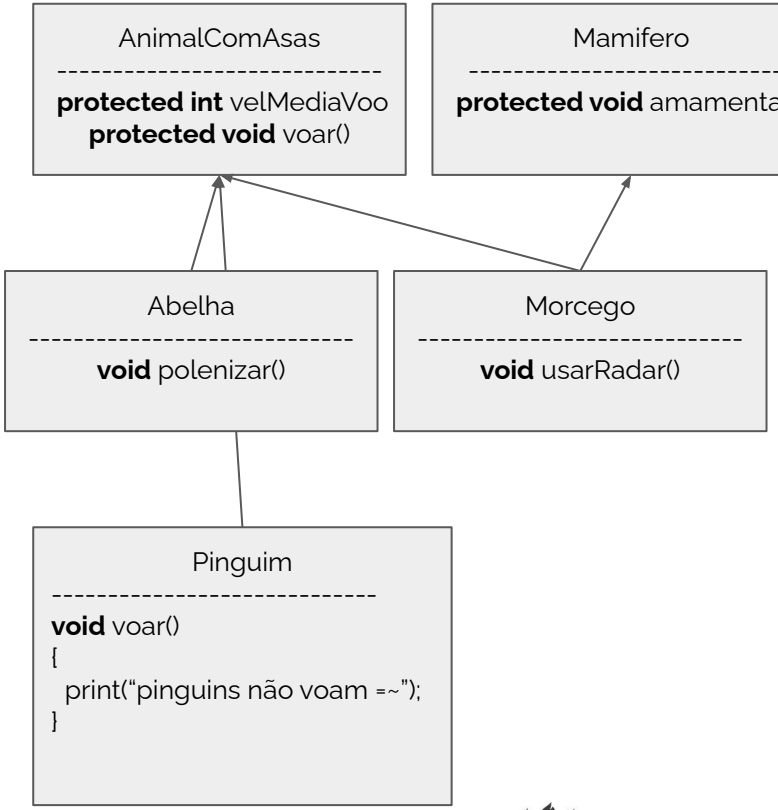
Ao contrário de C++, o Java não suporta **herança múltipla**, que ocorre quando uma classe é derivada de **mais de uma** superclasse direta



Modificadores de Acesso: Public x Private x Protected

Tudo em uma classe que consta como **private** é inacessível para outras classes, até aquelas que a herdam

Os membros **protected** de uma superclasse podem ser acessados por subclasses



Herança: Construtores em Subclasses

Instanciar um objeto de uma subclasse inicia uma **cadeia de chamadas do construtor** na qual ele, antes de realizar suas tarefas,

- usa *explicitamente* **super** para chamar um dos construtores em sua superclasse direta,
- ou então para chamar *implicitamente* o **construtor padrão** ou sem argumentos da superclasse.

Exercício

Inheritance in Java - GeeksforGeeks

```java
class A {
    public void print_A() { System.out.println("Class A"); }
}

class B extends A {
    public void print_B() { System.out.println("Class B"); }
}

class C extends A {
    public void print_C() { System.out.println("Class C"); }
}

class D extends A {
    public void print_D() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
        obj_D.print_D();
    }
}
```

Console:

```
Class A
Class B
Class A
Class C
Class A
Class D
```

Exercício

Inheritance in Java - GeeksforGeeks

```java
class A {
    public void print_() { System.out.println("Class A"); }
}

class B extends A {
    public void print_() { System.out.println("Class B"); }
}

class C extends A {
    public void print_() { System.out.println("Class C"); }
}

class D extends A {
    public void print_() { System.out.println("Class D"); }
}

// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_();

        C obj_C = new C();
        obj_C.print_();

        D obj_D = new D();
        obj_D.print_();

    }
}
```

Console:

```
Class B
Class C
Class D
```

Polimorfismo

O que significa polimorfismo?

Várias formas

Polimorfismo

O polimorfismo permite

"**programar no geral**"

em vez de

"**programar no específico**"

Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

Qualquer operação (por exemplo, calcular o perímetro ou a área) que pode ser realizada em um objeto Quadrilátero **também pode ser realizada** em um objeto Retângulo

Polimorfismo: Exemplos

Se a classe **Retângulo** é derivada da classe **Quadrilátero**, então um objeto Retângulo **é uma versão mais específica** de um objeto Quadrilátero

```java
abstract class Forma2D {
    public double area;
    abstract public void calcularArea();
}

class Retangulo extends Forma2D {
    public double altura;
    public double base;

    public Retangulo(double altura, double base)
    {
        this.altura = altura;
        this.base = base;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = altura * base;
    }

    public String toString(){
        return "Retangulo de altura: " + altura + " e base: " + base;
    }
}
```

```java
class Circulo extends Forma2D {
    public double raio;

    public Circulo(double raio)
    {
        this.raio = raio;
        this.calcularArea();
    }

    public void calcularArea(){
        this.area = Math.PI * Math.pow(raio, b: 2);
    }

    public String toString(){
        return "Criculo de raio: " + raio;
    }
}
```

```java
public static void main(String[] args) {

    Retangulo r1 = new Retangulo(altura: 1, base: 2);
    Retangulo r2 = new Retangulo(altura: 3, base: 5);
    Circulo c1 = new Circulo(raio: 7);
    Circulo c2 = new Circulo(raio: 9);

    ArrayList<Forma2D> formas = new ArrayList<Forma2D>();
    formas.add(r1);
    formas.add(r2);
    formas.add(c1);
    formas.add(c2);

    for (Forma2D forma : formas) {
        System.out.println(forma);
        System.out.println(">> a área dessa forma é: " + forma.area + "\n");
    }

    System.out.println(r1);
    System.out.println(r2);
    System.out.println(c1);
    System.out.println(c2);
```

Conceito de Hoje

Versionamento e Git

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Introdução

Problema

Motivação

História e Pessoas

Quando algo para de funcionar no meio do caminho...

Como saber
qual mudança
quebrou o código?

Como **recuperar**
uma versão anterior?

Versionamento

Email + ZIP

Google Drive

Google Docs

Git

Apresentação

Definições

Exemplos Ilustrativos

https://git-scm.com

git --local-branching-on-the-cheap

Search entire site...

Git is a free and open source distributed version control system designed to handle everything from small to very large projects with speed and efficiency.

Git is easy to learn and has a tiny footprint with lightning fast performance. It outclasses SCM tools like Subversion, CVS, Perforce, and ClearCase with features like cheap local branching, convenient staging areas, and multiple workflows.

About

The advantages of Git compared to other source control systems.

Documentation

Command reference pages, Pro Git book content, videos and other material.

Downloads

GUI clients and binary releases for all major platforms.

Community

Get involved! Bug reporting, mailing list, chat, development

Latest source Release

2.33.1

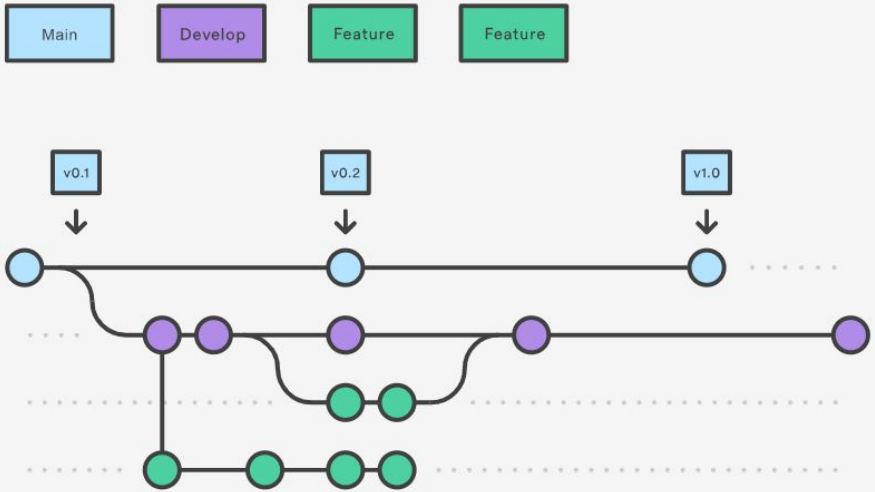
Release Notes (2021-10-12)

Download for Windows

Git (git-scm.com)

Feature branches

Each new feature should reside in its own branch, which can be pushed to the central repository for backup/collaboration. But, instead of branching off of `main`, `feature` branches use `develop` as their parent branch. When a feature is complete, it gets merged back into develop. Features should never interact directly with `main`.



Gitflow Workflow | Atlassian Git Tutorial

https://github.com

Search or jump to... /

Pull requests Issues Marketplace Explore

lsfcin

Repositories New

Find a repository...

- lsfcin / Programacao2
- lsfcin / InstagramDraft
- lams3 / GoThrough
- voxarlabs / 3DPO-Dataset
- lsfcin / haft
- lsfcin / air-guitar-framework
- lsfcin / aero-wings

Show more

Recent activity

When you take actions across GitHub, we'll provide links to that activity here.

All activity

hellyaxs started following you 6 days ago

Elias Vitor hellyaxs
Engenheiro da Computação
13 repositories 4 followers
Follow

arthuraaL pushed to voxarlabs/Virtual-Kinect on Sep 18

3 commits to main
435e37f Update README.md
30a0c83 Update README.md
1 more commit »

arthuraaL created a repository voxarlabs/Virtual-Kinect on Sep 18

voxarlabs/Virtual-Kinect
Updated Sep 18
Star

Universe 2021

Missed any Universe sessions? All content is available on demand now so you can view on your own time.

Watch the sessions on demand

GitHub

https://copilot.github.com

Your AI pair programmer

With GitHub Copilot, get suggestions for whole lines or entire functions right inside your editor.

Sign up >

sentiment.ts | write_sql.go | parse_expenses.py | addresses.rb

```
#!/usr/bin/env ts-node

import { fetch } from "fetch-h2";

// Determine whether the sentiment of text is positive
// Use a web service
async function isPositive(text: string): Promise<boolean> {
  const response = await fetch(`http://text-processing.com/api/sentiment/`, {
    method: "POST",
    body: `text=${text}`,
    headers: {
```

GitHub Copilot · Your AI pair programmer

Criando o Primeiro Repositório com Git

Crie uma conta no GitHub (github.com)

Crie um novo repositório no GitHub

Instale o Git (git-scm.com)

Execute o passo a passo disponível no GitHub

Clonando um Repositório

Acesse o GitHub (github.com)

Copie o código de clonagem e use no seu terminal Git localmente

Contribuindo para um Projeto com Git

Os comandos para realizar uma atualização de um código são

```
git add
git commit
git push
```

Segue um exemplo de uso

```
git add .
git commit -m ‘esta é uma mensagem explicando o que foi modificado’
git push origin main
```

coursera Explore What do you want to learn? For Enterprise For Students Log In Join for Free

Browse > Information Technology > Cloud Computing

This course is part of the **Google IT Automation with Python Professional Certificate**

Introduction to Git and GitHub

4.8 5,140 ratings | 98%

Google Career Certificates TOP INSTRUCTOR

Offered By Google

Enroll for Free
Starts Nov 17

Financial aid available

130,211 already enrolled

Introduction to Git and GitHub | Coursera

LUCAS SILVA FIGUEIREDO

UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Instruções

Versionamento do Código do Projeto

- Crie uma **conta** no GitHub (github.com)
- Crie um novo **repositório** no GitHub
- **Instale o Git** (git-scm.com)
- Execute o **passo a passo** disponível no GitHub
- Envie **todo o seu projeto** para o GitHub
  (com a adição de um arquivo **.gitignore** p/ códigos em java)
- No seu vídeo para a submissão grave a realização das etapas a seguir:
  . Faça uma **pequena alteração** no seu código
  . Execute/teste seu código com a alteração
  . Realize o fluxo de versionamento (**add**, **commit**, **push**)
  . Mostrem o **código atualizado** no repositório do **github**

Prática

Mini-projeto 1

Crie um programa que simula um "**livro interativo**", dando escolhas para o usuário, como por exemplo a direção que ele quer ir ("*esquerda*", "*direita*") em determinada situação.

Crie uma interação que permita pelo menos duas **escolhas consecutivas**, e que cada uma dessas escolhas leve para resultados diferentes.

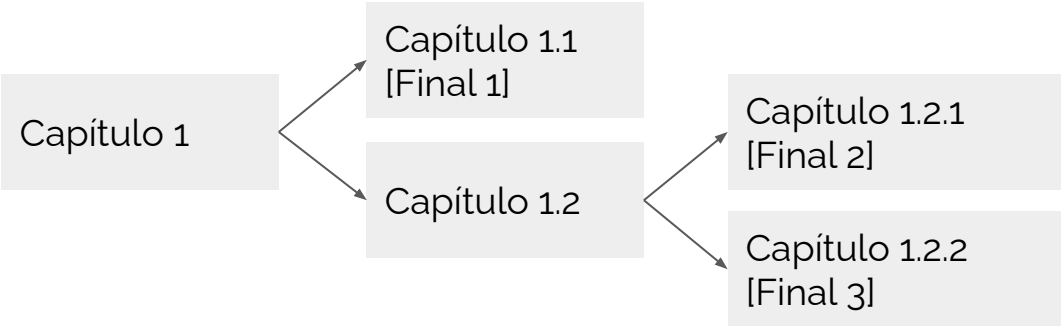
Devem existir pelo menos **três finais** diferentes.

Versione o projeto no GitHub.

```
Oi Neo, eu sou Morpheus, você está vivendo uma mentira. Você quer
ver a verdade? A pílula Azul deixará você na sua vida confortável
e irreal. A pílula Vermelha apresentará pra você um novo mundo.
Qual você escolhe?

Azul

Você escolheu ...
```



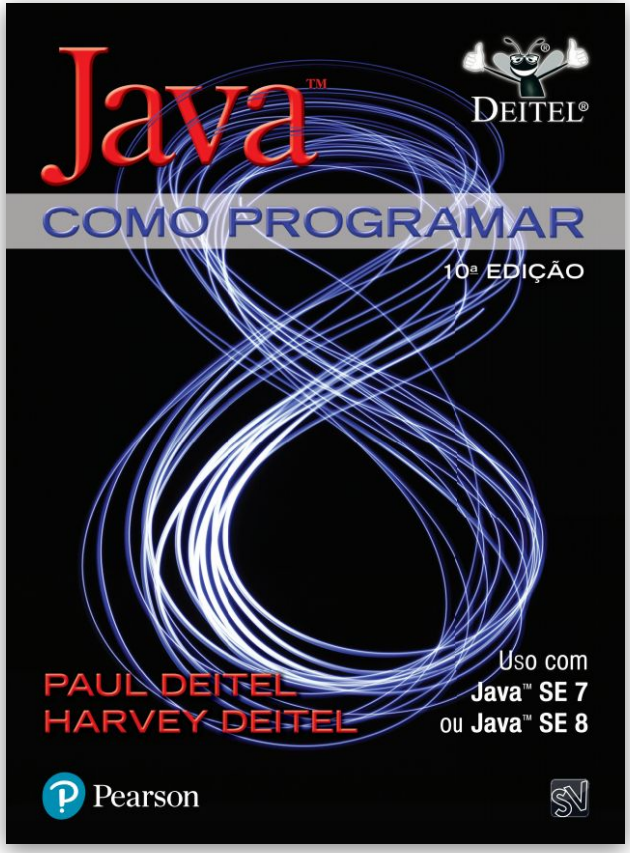
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Referências

Java - Como Programar (10a Edição)

9. Programação orientada a objetos: herança
    - 9.2 Superclasses e subclasses
    - 9.3 Membros protected
    - 9.5 Construtores em subclasses

10. Programação orientada a objetos: polimorfismo e interfaces
    - 10.4 Classes e métodos abstratos
    - 10.6 Atribuições permitidas entre variáveis ...
    - 10.9 Criando e utilizando interfaces

Ferramenta:

Eclipse: eclipse.org/downloads